

# **PREDLOG PROJEKTA IZ PREDMETA SISTEMI BAZIRANI NA ZNANJU**

2025/2026

**Tema: Ekspertski sistem za detekciju i prevenciju web napada (Dinamički WAF)**

Teodora Nikolić, SV66/2022

Softversko inženjerstvo i informacione tehnologije (SIIT)

## Cilj projekta

Cilj projekta je razvoj ekspertskog sistema za detekciju i prevenciju web napada, odnosno dinamičkog WAF sistema koji analizira HTTP zahteve u realnom vremenu i na osnovu definisanih pravila donosi odluke o njihovom propuštanju, blokiranju ili dodatnoj obradi. Sistem treba da prepozna potencijalno maliciozne obrasce ponašanja, kao što su SQL injection, XSS, path traversal, brute force pokušaji i skeniranje endpointa.

Rešenje će biti zasnovano na pravilima i kontekstima, čime se omogućava da sistem drugačije reaguje u normalnom režimu rada, tokom napada, u režimu održavanja ili u Zero Trust režimu. Pored detekcije pojedinačnih napada, sistem treba da omogući agregaciju događaja u vremenskim prozorima, automatsko povećavanje nivoa zaštite i prikaz bezbednosnih incidenata administratoru u realnom vremenu.

## Motivacija

Web aplikacije, mikroservisi i API sistemi danas predstavljaju osnovu velikog broja informacionih sistema, zbog čega su istovremeno i jedna od najčešćih meta sajber napada. Napadi na aplikativni sloj, kao što su SQL injection, XSS, path traversal, brute force pokušaji i automatizovano skeniranje endpointa, često ne mogu biti efikasno prepoznati korišćenjem tradicionalnih mrežnih firewall sistema, jer oni uglavnom rade na nivou IP adresa, portova i protokola.

Zbog toga se javlja potreba za sistemom koji analizira sam sadržaj HTTP zahteva i donosi odluke na osnovu značenja podataka koji se u zahtevu nalaze. Web Application Firewall (WAF) predstavlja važan mehanizam zaštite jer omogućava praćenje i filtriranje saobraćaja na aplikativnom nivou. Međutim, statički definisana pravila često nisu dovoljna, jer se obrasci napada menjaju, a sistem mora biti sposoban da se prilagođava različitim uslovima rada.

Motivacija za ovaj projekat je razvoj dinamičkog WAF sistema zasnovanog na znanju, koji bi administratorima omogućio lakše definisanje bezbednosnih pravila, automatsku detekciju sumnjivog ponašanja, reagovanje u realnom vremenu i pregled incidenata kroz administratorski interfejs. Korišćenjem ekspertskog sistema, konteksta, CEP obrade i agregacije događaja, sistem bi mogao da pruži fleksibilniji i inteligentniji pristup zaštiti web aplikacija.

## Pregled problema

Tradicionalni mrežni firewall sistemi uglavnom omogućavaju kontrolu pristupa na osnovu IP adresa, portova i protokola. Takav pristup je koristan za osnovnu mrežnu zaštitu, ali nije dovoljan za detekciju napada koji se odvijaju na aplikativnom sloju. Napadi kao što su SQL

injection, XSS, path traversal, brute force pokušaji i automatizovano skeniranje endpointa često se nalaze unutar HTTP zahteva, odnosno u URI putanji, query parametrima, zaglavljima ili telu zahteva. Zbog toga ih nije moguće pouzdano prepoznati samo analizom mrežnih karakteristika.

Dodatni problem predstavlja velika količina web saobraćaja koja u realnim sistemima neprestano pristiže. Jedan korisnik ili bot može generisati veliki broj zahteva u kratkom vremenskom periodu, dok koordinisani napadi mogu dolaziti sa više različitih IP adresa. Zbog toga sistem ne sme posmatrati svaki zahtev isključivo izolovano, već mora biti sposoban da prati obrasce ponašanja kroz vreme, agregira događaje i prepozna situacije koje ukazuju na napad.

Još jedan izazov je promenljivost sajber pretnji. Novi oblici napada, maliciozni obrasci i sumnjivi payload-i pojavljuju se često, pa sistem mora biti proširiv i konfigurabilan. Administrator treba da ima mogućnost da menja pragove osetljivosti, dodaje nova pravila i prilagođava ponašanje sistema različitim režimima rada, kao što su normalan režim, režim pod napadom, održavanje ili Zero Trust režim.

Zbog navedenih problema, cilj je razviti dinamički WAF sistem zasnovan na pravilima, koji će analizirati HTTP zahteve u realnom vremenu, prepoznavati potencijalne napade, automatski donositi odluke o propuštanju ili blokiranju zahteva i omogućiti administratoru pregled incidenata i stanja sistema. Korišćenjem ekspertskog sistema, CEP obrade i agregacije događaja, sistem može da obezbedi fleksibilniju i inteligentniju zaštitu web aplikacija.

## Metodologija rada

Sistem će obrađivati dve osnovne vrste ulaza: real-time podatke o HTTP zahtevima i konfiguracione podatke koje unosi administrator. Real-time podaci dolaze sa web servera ili reverse proxy komponente, na primer Nginx-a, koji prosleđuje informacije o pristiglim zahtevima WAF sistemu. Svaki HTTP zahtev biće transformisan u događaj, odnosno objekat RequestEvent, nad kojim će se izvršavati pravila ekspertskog sistema.

Od podataka iz HTTP zahteva očekuju se sledeći parametri:

- IP adresa izvora,
- HTTP metoda, na primer GET, POST, PUT ili DELETE,
- URI putanja,
- HTTP zaglavlja,
- query parametri,
- telo zahteva, odnosno payload,
- vreme pristizanja zahteva,
- status prethodnih zahteva iste IP adrese, ako postoji.

Pored real-time podataka, administrator unosi konfiguracione podatke koji određuju ponašanje sistema. Konfiguracija obuhvata pragove osetljivosti, dozvoljene formate zahteva, trajanje blokade IP adrese, maksimalan broj zahteva u određenom vremenskom periodu i pravila koja se koriste za detekciju napada.

Obrada zahteva započinje prijemom RequestEvent događaja. Drools engine zatim proverava skup aktivnih pravila u skladu sa trenutno aktivnim kontekstom sistema. Ako zahtev sadrži sumnjive obrasce, kao što su SQL ključne reči, script tagovi, path traversal sekvence ili neuobičajen broj pokušaja pristupa, sistem generiše odgovarajući bezbednosni događaj. Na osnovu ozbiljnosti događaja i prethodnog ponašanja iste IP adrese, sistem može povećati threat score, blokirati pojedinačni zahtev, privremeno zabraniti pristup IP adresi ili poslati upozorenje administratoru.

Izlazi sistema predstavljaju akcije kojima se upravlja zaštitom web aplikacije:

- Allow — zahtev se propušta ka backend aplikaciji,
- Drop — zahtev se blokira,
- Ban — IP adresa se blokira na određeni vremenski period,
- Alert — administrator dobija upozorenje o potencijalnom napadu,
- Log — incident se čuva radi kasnije analize i izveštavanja.

Kako bi sistem bio prilagodljiv različitim situacijama, uvodi se koncept konteksta. Kontekst predstavlja režim rada sistema i određuje koliko strogo će se pravila primenjivati. Na primer, u normalnom režimu sistem koristi standardne pragove, dok u režimu napada povećava osetljivost i brže blokira sumnjive zahteve. Na taj način isti skup pravila može davati različite odluke u zavisnosti od trenutnog stanja sistema.

## Konteksti sistema

Kako bi sistem mogao da se prilagodi različitim bezbednosnim situacijama, uvodi se koncept konteksta. Kontekst predstavlja trenutno stanje ili režim rada WAF sistema i utiče na to koja pravila će biti aktivna, koliko strogi će biti pragovi detekcije i koje akcije će sistem preduzimati nakon detekcije sumnjivog ponašanja.

Za svaki kontekst moguće je konfigurisati sledeće parametre:

- maksimalan broj zahteva po minuti,
- dozvoljene HTTP metode,
- dozvoljene formate payload-a,
- nivo osetljivosti na specijalne karaktere u parametrima,
- maksimalan dozvoljeni threat score za jednu IP adresu,
- vreme trajanja privremene IP blokade,
- broj dozvoljenih neuspešnih login pokušaja,
- dozvoljene ili zabranjene URI putanje,

- liste pouzdanih IP adresa ili subnet-a.

U sistemu će postojati sledeći predefinisani konteksti:

- **Normal\_Traffic** predstavlja standardni režim rada sistema. U ovom režimu se primenjuju uobičajena pravila zaštite, kao što su detekcija SQL injection, XSS i path traversal pokušaja. Pragovi su podešeni tako da se spreči veliki broj lažno pozitivnih detekcija, ali da se ipak prepoznaju očigledno sumnjivi zahtevi.
- **Under\_Attack** predstavlja režim povećane zaštite. Aktivira se kada sistem detektuje veći broj incidenata u kratkom vremenskom periodu, na primer veliki broj blokiranih zahteva, brute force pokušaja ili skeniranja endpointa. U ovom kontekstu pragovi se smanjuju, sistem brže povećava threat score i strože reaguje na sumnjive obrasce ponašanja.
- **Maintenance** predstavlja režim održavanja sistema. U ovom režimu pristup određenim endpointima može biti dozvoljen samo internim IP adresama ili administratorima. Svi zahtevi koji dolaze izvan definisanih pouzdanih mreža mogu biti blokirani ili označeni kao sumnjivi.
- **Zero\_Trust** predstavlja najrestriktivniji režim rada sistema. Aktivira se u slučaju ozbiljnog ili koordinisanog napada. U ovom režimu se podrazumeva da nijedan zahtev nije pouzdan dok se ne proverí. Sistem blokira sve zahteve koji ne dolaze iz pouzdanih izvora ili sadrže bilo kakve sumnjive elemente, kao što su nepoznata zaglavlja, neuobičajeni User-Agent, specijalni karakteri u parametrima ili pristup osetljivim putanjama.

Kontekst može biti promenjen ručno od strane administratora ili automatski, kao posledica okidanja pravila. Na primer, ako sistem u vremenskom prozoru od jednog minuta detektuje veliki broj blokiranih IP adresa, može automatski promeniti globalni kontekst iz Normal\_Traffic u Zero\_Trust i poslati upozorenje administratoru.

## Baza znanja

Baza znanja predstavlja centralni deo ekspertskog sistema. U njoj se nalaze pravila kojima se opisuje ponašanje WAF sistema prilikom obrade HTTP zahteva. Pravila se izvršavaju nad događajima tipa RequestEvent, kao i nad izvedenim bezbednosnim događajima koji nastaju tokom rada sistema, na primer Potential\_SQL\_Injection, Potential\_XSS, Brute\_Force\_Attempt ili Malicious\_Payload\_Detected.

Legenda:

- K — trenutno aktivni kontekst sistema,
- C[K] — konfiguracija za konkretni kontekst,
- RequestEvent — događaj koji predstavlja jedan HTTP zahtev,
- ThreatScore(IP) — trenutni skor sumnjivosti za određenu IP adresu,
- dispatch(eventName) — generisanje novog bezbednosnog događaja,
- logThreat() — čuvanje incidenta u sistemu,

- blockRequest() — blokiranje konkretnog HTTP zahteva,
- banIpAddress(IP) — privremena blokada IP adrese,
- alertAdmin() — slanje upozorenja administratoru.

### **Primeri osnovnih pravila za detekciju napada:**

```

when
    (K == Normal_Traffic or K == Under_Attack)
    and containsSqlKeywords(query_params)
then
    logThreat()
    and dispatch(Potential_SQL_Injection)
    and incrementThreatScore(IP)

when
    (K == Normal_Traffic or K == Under_Attack)
    and containsScriptTags(payload)
then
    logThreat()
    and dispatch(Potential_XSS)
    and incrementThreatScore(IP)

when
    any(K)
    and hasPathTraversalSequence(uri)
then
    logThreat()
    and dispatch(Potential_Path_Traversal)
    and incrementThreatScore(IP)

when
    K == Zero_Trust
    and notFromTrustedSubnet(IP)
then
    blockRequest()
    and dispatch(Unauthorized_Access_Attempt)
    and alertAdmin()

when
    any(K)
    and hasSuspiciousUserAgent(headers)
then
    logThreat()
    and dispatch(Suspicious_Bot_Detected)
    and incrementThreatScore(IP)

```

### **Primeri CEP pravila za detekciju ponašanja kroz vreme:**

```
when
    more than C[K](max_failed_logins_per_minute) Failed_Login events
    from same IP in 1 minute
then
    dispatch(Brute_Force_Attempt)
    and banIpAddress(IP)
    and alertAdmin()

when
    more than C[K](max_not_found_requests_per_minute) Not_Found_Request
events
    from same IP in 1 minute
then
    dispatch(Endpoint_Scanning)
    and incrementThreatScore(IP)

when
    more than C[K](max_requests_per_minute) RequestEvent events
    from same IP in 1 minute
then
    dispatch(Rate_Limit_Exceeded)
    and blockRequest()
    and incrementThreatScore(IP)
```

### **Primeri pravila za donošenje odluka:**

```
when
    Potential_SQL_Injection
    or Potential_XSS
    or Potential_Path_Traversal
then
    dispatch(Malicious_Payload_Detected)

when
    Malicious_Payload_Detected
    and isRequestFrom(IP)
then
    blockRequest()
    and incrementThreatScore(IP)

when
    ThreatScore(IP) > C[K](max_allowed_score)
then
    banIpAddress(IP)
```

```
and alertAdmin()
```

### **Primer pravila za automatsku promenu konteksta:**

```
when
    Number(intValue > C[K](max_banned_ips_per_minute)) from
    accumulate(
        $b: BannedIpEvent() over window:time(1m),
        count($b)
    )
then
    changeGlobalContext(Zero_Trust)
    and alertAdmin("Massive attack detected. Switching to Zero Trust.")
    and dispatch(Global_Under_Attack)

when
    Global_Under_Attack
    and RequestEvent(hasAnySuspiciousHeaders())
then
    dropRequest()
    and logThreat()
```

## Primer forward chaining-a

Forward chaining se koristi za automatsko izvođenje novih zaključaka na osnovu pristiglih događaja i prethodno definisanih pravila. Sistem ne donosi odluku samo na osnovu jednog pravila, već se zaključci mogu nadovezivati jedan na drugi.

**Na primer, kada korisnik pošalje HTTP zahtev koji u query parametrima sadrži SQL ključne reči, sistem najpre detektuje potencijalni SQL injection napad:**

```
when
    containsSqlKeywords(query_params)
then
    dispatch(Potential_SQL_Injection)
    and logThreat()
    and incrementThreatScore(IP)
```

**Nakon toga, novoizvedeni događaj Potential\_SQL\_Injection može aktivirati drugo pravilo koje zaključuje da zahtev sadrži maliciozni payload:**

```
when
    Potential_SQL_Injection
    or Potential_XSS
    or Potential_Path_Traversal
```

```
then
    dispatch(Malicious_Payload_Detected)
```

**Kada se izvede zaključak `Malicious_Payload_Detected`, sistem blokira konkretan zahtev i dodatno povećava threat score IP adrese:**

```
when
    Malicious_Payload_Detected
    and isRequestFrom(IP)
then
    blockRequest()
    and incrementThreatScore(IP)
```

**Ukoliko ista IP adresa nastavi da šalje sumnjive zahteve, njen threat score može preći maksimalno dozvoljenu vrednost za trenutni kontekst. Tada se aktivira pravilo za privremenu blokadu IP adrese:**

```
when
    ThreatScore(IP) > C[K](max_allowed_score)
then
    banIpAddress(IP)
    and alertAdmin()
```

**Forward chaining omogućava i reakciju na koordinisane napade. Ako sistem u kratkom vremenskom periodu detektuje veliki broj blokiranih IP adresa, zaključuje da je sistem potencijalno pod masovnim napadom. Tada se automatski menja globalni kontekst u `Zero_Trust` režim:**

```
when
    Number(intValue > C[K](max_banned_ips_per_minute)) from
    accumulate(
        $b: BannedIpEvent() over window:time(1m),
        count($b)
    )
then
    changeGlobalContext(Zero_Trust)
    and alertAdmin("Massive attack detected. Switching to Zero
Trust.")
    and dispatch(Global_Under_Attack)
```

Na ovaj način sistem od jednog sumnjivog HTTP zahteva može doći do složenije odluke, kao što su blokiranje zahteva, povećanje threat score-a, privremena zabrana IP adrese i automatsko podizanje nivoa zaštite celog sistema.

# Agregacija podataka i CEP obrada

Podaci o HTTP zahtevima biće predstavljeni u obliku događaja. Svaki pristigli zahtev generiše jedan RequestEvent, dok se tokom rada sistema mogu generisati i dodatni bezbednosni događaji, kao što su Potential\_SQL\_Injection, Potential\_XSS, Failed\_Login, Blocked\_Request, BannedIpEvent ili Endpoint\_Scanning. Pošto web aplikacije u realnim uslovima mogu primati veliki broj zahteva u kratkom vremenskom periodu, nije dovoljno analizirati svaki zahtev izolovano. Potrebno je pratiti obrasce ponašanja kroz vreme.

Za obradu događaja koristiće se CEP, odnosno Complex Event Processing. CEP omogućava da sistem prepozna složene događaje na osnovu više pojedinačnih događaja koji se dešavaju u određenom vremenskom prozoru. Na primer, jedan neuspešan pokušaj prijave ne mora biti opasan, ali veliki broj neuspešnih pokušaja sa iste IP adrese u kratkom periodu može ukazivati na brute force napad.

Događaji će se agregirati u vremenskim prozorima različite rezolucije:

- 10 sekundi,
- 1 minut,
- 15 minuta,
- 1 sat,
- 24 sata.

Nad događajima će se primenjivati agregacione funkcije kao što su:

- count — broj događaja određenog tipa,
- sum — ukupan broj grešaka ili blokiranih zahteva,
- min i max — najmanja i najveća vrednost posmatranog parametra,
- average — prosečna vrednost, na primer prosečan broj zahteva po IP adresi.

## Primer CEP pravila za brute force napad:

```
when
    more than 50 Failed_Login events
    from same IP
    over window:time(1m)
then
    dispatch(Brute_Force_Attempt)
    and banIpAddress(IP)
    and alertAdmin()
```

## Primer CEP pravila za rate limiting:

```
when
    more than C[K](max_requests_per_minute) RequestEvent events
    from same IP
    over window:time(1m)
then
```

```
dispatch(Rate_Limit_Exceeded)
and blockRequest()
and incrementThreatScore(IP)
```

### **Primer CEP pravila za skeniranje endpointa:**

```
when
    more than 30 Not_Found_Request events
    from same IP
    over window:time(1m)
then
    dispatch(Endpoint_Scanning)
    and incrementThreatScore(IP)
    and alertAdmin()
```

Agregacija je važna i zbog efikasnog korišćenja radne memorije. Sirovi HTTP događaji sa najnižom rezolucijom neće se trajno čuvati u radnoj memoriji ekspertskeg sistema. Nakon što se izvrši agregacija u viši vremenski prozor, pojedinačni događaji mogu se ukloniti iz radne memorije, dok se čuvaju samo agregirani rezultati potrebni za dalju analizu i izveštavanje. Na taj način se smanjuje opterećenje sistema i sprečava nepotrebno gomilanje događaja.

Vremena agregacije biće strogo poravnata, na primer na 12:00, 12:15, 12:30 i 12:45, kako bi se olakšalo poređenje podataka i kasnija analitika. Agregirani podaci biće organizovani hijerarhijski, tako da se podaci niže rezolucije mogu koristiti za izračunavanje agregacija višeg nivoa. Ovakva organizacija omogućava efikasnije izveštavanje i predstavlja osnovu za backward chaining analizu uzroka bezbednosnih incidenata.

## **Izveštaji (primer backward chaining-a)**

**Izveštaji** predstavljaju važan deo WAF sistema, jer administratoru omogućavaju analizu incidenata nakon što se napad dogodio. Pored prikaza osnovnih statistika, sistem treba da omogući i objašnjenje donetih odluka. Zbog toga se backward chaining može koristiti za pronalaženje uzroka određenog zaključka, na primer zašto je zahtev blokiran ili zašto je određena IP adresa privremeno zabranjena.

**Backward chaining** se koristi tako što sistem kreće od zaključka i proverava koja pravila i koji prethodni događaji su doveli do njega. Na primer, ako administrator želi da proveri zašto je nastao događaj `Malicious_Payload_Detected`, sistem može unazad analizirati da li je pre toga detektovan `Potential_SQL_Injection`, `Potential_XSS` ili `Potential_Path_Traversal` događaj.

### Primer toka backward chaining-a:

Cilj → **Zašto je zahtev blokiran?**

Sistem proverava:

- Da li je zahtev blokiran zato što je nastao događaj `Malicious_Payload_Detected`?
- Ako jeste, šta je izazvalo `Malicious_Payload_Detected`?
- Da li je pre toga detektovan `Potential_SQL_Injection`?
- Da li su query parametri sadržali SQL ključne reči?
- Koja IP adresa je poslala zahtev?
- Koja URI putanja je gađana?
- Koji payload ili header je bio sumnjiv?

Na ovaj način administrator ne dobija samo informaciju da je zahtev blokiran, već i objašnjenje celog lanca zaključivanja. Na primer, sistem može prikazati da je zahtev blokiran zato što je query parametar sadržao SQL ključne reči, zatim je generisan događaj `Potential_SQL_Injection`, nakon toga `Malicious_Payload_Detected`, a zatim je aktivirano pravilo za `blockRequest` akciju.

U sistemu će postojati više tipova izveštaja:

- 1. Root-cause izveštaj** - Ovaj izveštaj prikazuje uzrok određenog bezbednosnog incidenta. Administrator može izabrati konkretan događaj, na primer ban IP adrese, blokiran zahtev ili brute force upozorenje, a sistem zatim prikazuje događaje i pravila koja su dovela do tog zaključka. Ovaj izveštaj je koristan za proveru da li je sistem pravilno reagovao i da li je incident zaista bio maliciozan.
- 2. Top Attackers izveštaj** - Ovaj izveštaj prikazuje IP adrese ili mrežne opsege koji su generisali najveći broj sumnjivih događaja u izabranom vremenskom periodu. Moguće je filtrirati podatke po tipu napada, na primer SQL injection, XSS, brute force, endpoint scanning ili path traversal.
- 3. Attack Types izveštaj** - Ovaj izveštaj prikazuje najčešće tipove napada u određenom periodu. Administrator može videti da li su u sistemu najčešći brute force pokušaji, XSS napadi, SQL injection pokušaji ili skeniranje endpointa. Ovakav izveštaj pomaže pri podešavanju pravila i pragova osetljivosti.
- 4. Context Changes izveštaj** - Ovaj izveštaj prikazuje kada je sistem promenio kontekst rada, na primer iz `Normal_Traffic` u `Under_Attack` ili `Zero_Trust`. Za svaku promenu prikazuje se razlog promene, broj incidenata u vremenskom prozoru i pravilo koje je aktiviralo promenu konteksta.
- 5. Blocked Requests izveštaj** - Ovaj izveštaj prikazuje sve blokirane HTTP zahteve, zajedno sa IP adresom, metodom, URI putanjom, vremenom zahteva, razlogom blokiranja i akcijom koju je sistem preduzeo.

## Realtime data

Blagovremena reakcija je veoma važna u oblasti sajber bezbednosti, jer napadi na web aplikacije često mogu trajati veoma kratko, ali izazvati ozbiljne posledice. Zbog toga sistem treba da omogući administratoru uvid u trenutno stanje web saobraćaja, aktivne pretnje i odluke koje WAF donosi u realnom vremenu.

Podaci koji nastaju u Drools engine-u biće prosleđivani administratorskom interfejsu, koji će biti implementiran kao Angular aplikacija. Dashboard će prikazivati najvažnije informacije o trenutnom stanju sistema, kao što su broj pristiglih zahteva, broj blokiranih zahteva, broj aktivnih incidenata, trenutno aktivni kontekst i lista IP adresa koje su privremeno blokirane.

Na dashboard-u će biti prikazani sledeći elementi:

- broj HTTP zahteva u poslednjem minutu,
- broj blokiranih zahteva u poslednjem satu,
- broj trenutno aktivnih IP blokada,
- trenutno aktivni kontekst sistema,
- live feed bezbednosnih događaja,
- najčešći tipovi napada,
- IP adrese sa najvećim threat score vrednostima,
- upozorenja o kritičnim incidentima.

Live feed će prikazivati događaje čim se oni dogode. Na primer, administrator može odmah videti da je detektovan `Potential_SQL_Injection`, `Brute_Force_Attempt`, `Endpoint_Scanning` ili `Global_Under_Attack` događaj. Za svaki događaj prikazivaće se vreme, IP adresa, tip napada, URI putanja, trenutni kontekst i akcija koju je sistem preduzeo.

Posebno važan deo dashboard-a biće prikaz promene konteksta. Ako sistem automatski promeni režim rada iz `Normal_Traffic` u `Under_Attack` ili `Zero_Trust`, administrator će dobiti jasno obaveštenje o razlogu promene. Na primer, sistem može prikazati da je kontekst promenjen zato što je u poslednjem minutu detektovan veliki broj blokiranih IP adresa.

Za kritične događaje, kao što su pokušaj napada na administratorski nalog, veliki broj brute force pokušaja ili koordinisani napad sa više IP adresa, sistem će slati upozorenja administratoru u realnom vremenu. Upozorenja mogu biti prikazana u aplikaciji kroz notifikacije, a dodatno se mogu čuvati u bazi radi kasnije analize.

Realtime prikaz omogućava administratoru da brzo reaguje, ručno promeni kontekst sistema, proveriti razloge blokiranja zahteva i prilagodi pravila ukoliko se pojavi novi obrazac napada.

## Domenski specifičan jezik (DSL)

Jedan od problema pri implementaciji bezbednosnih pravila jeste to što stručnjaci za sajber bezbednost ne moraju nužno poznavati **Drools** ili **Java** sintaksu. Sa druge strane, upravo oni najbolje poznaju obrasce napada, sumnjive payload-e, IP opsege koje treba blokirati i pravila koja treba primeniti u određenim situacijama. Zbog toga se uvodi domenski specifičan jezik, odnosno DSL, koji omogućava jednostavnije definisanje pravila nalik prirodnom jeziku.

**DSL** enkapsulira složenost Drools pravila i omogućava administratoru da pravila piše kroz razumljive izraze. Na taj način se smanjuje mogućnost greške, ubrzava dodavanje novih

pravila i omogućava lakše održavanje sistema. Pravila napisana u DSL-u biće prevedena u odgovarajuća Drools pravila koja se zatim izvršavaju u ekspertskom sistemu.

Primeri DSL sintakse:

- Provera HTTP metode: method is {methodType}
- Provera URI putanje: uri contains {pattern} → uri contains "/admin"
- Provera query parametara: query contains {keyword} → query contains "UNION SELECT"
- Provera tela zahteva: body contains {pattern} → body contains "<script>"
- Provera IP adrese: ip is (not) from trusted subnet
- Provera broja događaja u vremenskom prozoru: more than {number} {event} from same IP in {time} min → more than 50 Failed\_Login from same IP in 1 min
- Akcije sistema: block request, ban IP for {time} minutes, log as {threat\_name}, alert admin, change context to {context\_name}

#### **Primer kompletnog DSL pravila:**

```
when
    method is POST
    and body contains "<script>"
then
    block request
    and log as XSS_ATTACK
    and alert admin
```

#### **Primer pravila za brute force napad:**

```
when
    more than 50 Failed_Login from same IP in 1 min
then
    ban IP for 30 minutes
    and log as BRUTE_FORCE_ATTACK
    and alert admin
```

#### **Primer pravila za Zero Trust režim:**

```
when
    context is Zero_Trust
    and ip is not from trusted subnet
then
    block request
    and log as UNAUTHORIZED_ACCESS_ATTEMPT
```

#### **Primer pravila za SQL injection:**

```
when
    query contains "UNION SELECT"
    or query contains "' OR '1'='1"
then
    block request
    and log as SQL_INJECTION_ATTEMPT
    and increment threat score
```

Na ovaj način administrator može dodavati i menjati bezbednosna pravila bez direktnog pisanja Drools sintakse. Sistem zatim prevodi DSL pravila u formalna pravila ekspertskeg sistema i primenjuje ih nad pristiglim HTTP događajima.

## Proširivost pravila (Template)

Sajber pretnje se neprestano menjaju, zbog čega WAF sistem mora biti proširiv i lako prilagodljiv. Statički definisana pravila nisu dovoljna za dugoročnu zaštitu, jer se pojavljuju novi maliciozni payload-i, novi obrasci automatizovanih napada, nove IP adrese botnet mreža i novi načini zaobilazjenja postojećih filtera.

Zbog toga sistem omogućava proširivanje baze znanja korišćenjem Drools template-a. Template predstavlja šablon pravila koji se može instancirati sa konkretnim parametrima. Administrator ne mora ručno da piše celo Drools pravilo, već kroz korisnički interfejs unosi vrednosti kao što su naziv pretnje, ključna reč, IP opseg, URI putanja, dozvoljena metoda ili akcija koju sistem treba da izvrši.

### **Primer template-a za blokiranje maliciozne ključne reči:**

Template: BlockMaliciousKeyword

Parametri:

- keyword,
- threatName,
- action.

### **Opšti oblik pravila:**

```
when
    request contains {keyword}
then
    {action}
    and log as {threatName}
    and increment threat score
```

### **Primer instance template-a:**

```
keyword = "UNION SELECT"
threatName = "SQL_INJECTION_ATTEMPT"
```

```
action = "block request"
```

Na osnovu ovih parametara sistem generiše konkretno pravilo koje blokira zahteve koji sadrže zadatu SQL injection ključnu reč.

#### **Primer template-a za blokiranje IP opsega:**

Template: BlockIpRange

Parametri:

- ipRange,
- reason,
- banDuration.

#### **Opšti oblik pravila:**

```
when
    IP is from {ipRange}
then
    ban IP for {banDuration}
    and log as {reason}
    and alert admin
```

#### **Primer instance:**

```
ipRange = "192.168.10.0/24"
reason = "KNOWN_MALICIOUS_RANGE"
banDuration = "60 minutes"
```

#### **Primer template-a za zaštitu administratorskih endpointa:**

Template: ProtectSensitiveEndpoint

Parametri:

- uriPattern,
- allowedSubnet,
- threatName.

#### **Opšti oblik pravila:**

```
when
    uri contains {uriPattern}
    and IP is not from {allowedSubnet}
then
    block request
    and log as {threatName}
    and alert admin
```

#### **Primer instance:**

```
uriPattern = "/admin"  
allowedSubnet = "10.0.0.0/8"  
threatName = "UNAUTHORIZED_ADMIN_ACCESS"
```

Korišćenjem template-a administrator može brzo da doda nova pravila bez izmene izvornog koda aplikacije. Nakon unosa parametara kroz korisnički interfejs, sistem generiše odgovarajuće Drools pravilo i primenjuje ga u radnoj memoriji. Na taj način se baza znanja može proširivati bez restartovanja Spring Boot aplikacije, što je važno za sisteme koji treba da rade neprekidno.

## Arhitektura sistema

Sistem je zamišljen kao višeslojna aplikacija koja se sastoji od reverse proxy komponente, backend WAF servisa, ekspertskog sistema, baze podataka i administratorskog frontend interfejsa.

**Prvi sloj** sistema predstavlja reverse proxy, na primer Nginx, koji prima HTTP zahteve od korisnika, botova ili potencijalnih napadača. Umesto da se zahtev odmah prosledi backend aplikaciji, Nginx prosleđuje relevantne podatke o zahtevu WAF servisu. Ti podaci uključuju IP adresu izvora, HTTP metodu, URI putanju, zaglavlja, query parametre, payload i vreme pristizanja zahteva.

**Drugi sloj** predstavlja Spring Boot WAF servis. Njegova uloga je da primi podatke o HTTP zahtevu, transformiše ih u događaj tipa RequestEvent i prosledi ih Drools engine-u. Spring Boot servis takođe upravlja konfiguracijama konteksta, čuva incidente, vodi evidenciju o blokiranim IP adresama i omogućava komunikaciju sa administratorskim interfejsom.

**Centralni deo sistema** predstavlja Drools engine, odnosno ekspertski sistem. On izvršava pravila iz baze znanja nad pristiglim događajima. Na osnovu trenutno aktivnog konteksta i sadržaja zahteva, Drools donosi zaključke kao što su Potential\_SQL\_Injection, Potential\_XSS, Brute\_Force\_Attempt, Malicious\_Payload\_Detected ili Global\_Under\_Attack. Na osnovu tih zaključaka sistem odlučuje da li će zahtev biti propušten, blokirano, da li će IP adresa biti privremeno zabranjena ili će administrator dobiti upozorenje.

**Baza podataka** koristi se za čuvanje konfiguracija, definisanih pravila, konteksta, incidenata, agregiranih podataka i istorije blokiranih IP adresa. Na ovaj način administrator može kasnije da pregleda izveštaje, analizira napade i proveriti zašto je određena odluka doneta.

**Frontend deo sistema** biće implementiran kao Angular aplikacija. Administrator kroz ovu aplikaciju može da prati stanje sistema u realnom vremenu, pregleda live feed pretnji, vidi trenutno aktivni kontekst, proveriti blokirane IP adrese, analizira izveštaje i dodaje nova pravila kroz DSL ili template mehanizam.

Tok obrade jednog zahteva može se opisati sledećim koracima:

1. Korisnik ili bot šalje HTTP zahtev ka web aplikaciji.
2. Reverse proxy prima zahtev i prosleđuje njegove podatke WAF servisu.
3. Spring Boot WAF servis kreira RequestEvent događaj.
4. Drools engine proverava pravila nad pristiglim događajem.
5. Ako zahtev nije sumnjiv, sistem donosi odluku Allow i zahtev se prosleđuje backend aplikaciji.
6. Ako je zahtev sumnjiv, sistem može izvršiti Drop, Ban, Alert ili Log akciju.
7. Incident se čuva u bazi i prikazuje administratoru na dashboard-u.
8. Ako se detektuje veći broj incidenata, sistem može automatski promeniti kontekst, na primer iz Normal\_Traffic u Under\_Attack ili Zero\_Trust.